Calidad del Software Actividad: Caso Estudio Integrador.

Desarrollo de Calculadora usando metodología RUP.

Disciplina: 4. Implementación.

Estudiante: Diego Andrés Díaz.

Cc 80032909

Docente: Ing Leonel Andrés Lópe Arias.

Institución: Politécnico Internacional.

Fecha: Noviembre 2026.

1. Introducción.

La implementación convierte el diseño en código ejecutable.

En la fase de Transición el código está completo, integrado y verificado con pruebas automáticas. El código fuente completo se incluye en la sección 9 y en la carpeta del proyecto.

Trabajo a realizar (Tabla 4 del caso)	Artefacto resultante	Sección
Implementar la arquitectura	Modelo de implementación	3
Integrar el sistema	Componentes	4
Implementar los subsistemas	Subsistemas de implementación	5
Implementar las clases	Interfaz	6
Realizar prueba de unidad	Descripción de arquitectura (vista del modelo de implementación)	7
	Integración de construcciones	8

2. Entorno de desarrollo.

Elemento	Detalle
Lenguaje	Python 3 (código escrito para 3.8 o superior; las pruebas de este documento se ejecutaron con 3.12.3)
Kit de desarrollo y librerías	Biblioteca estándar de Python: `tkinter` (interfaz), `math` y `unittest` (pruebas). No requiere instalar paquetes.
IDE	Visual Studio Code con la extensión de Python
Plataforma	Windows 11
Control de versiones	Git y repositorio en GitHub
Ejecución	`python main.py` desde la carpeta raíz del proyecto

3. Modelo de implementación.

Estructura de archivos del proyecto, que refleja la arquitectura MVC: calculadora-rup/
|-- main.py punto de entrada
|-- README.md
|-- .gitignore
|-- calculadora/
| |-- __init__.py
| |-- modelo.py Modelo (clase Calculadora)
| |-- controlador.py Controlador (clase Controlador)
| `-- vista.py Vista (clase Vista, Tkinter)
|-- pruebas/
| |-- __init__.py
| |-- test_modelo.py
| |-- test_controlador.py
| `-- test_vista.py `-- docs/ artefactos en PDF de cada disciplina.

4. Componentes.

Componente	Archivo	Líneas	Clase de diseño	Responsabilidad
Arranque	main.py	3	—	Crea la Vista y la inicia.
Vista	calculadora/vista.py	40	Vista	Formulario con dos campos, dos botones y el campo de resultado.
Controlador	calculadora/controlador.py	25	Controlador	Valida datos, aplica la operación y da formato al resultado.
Modelo	calculadora/modelo.py	6	Calculadora	Fórmulas de suma y multiplicación.

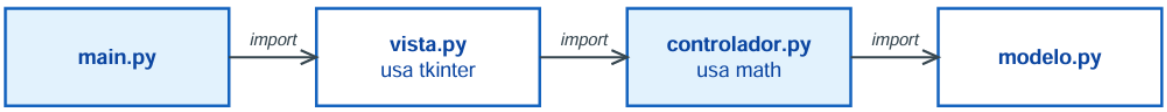
5. Subsistemas de implementación.

Subsistema	Contenido	Descripción
Paquete `calculadora`	vista.py, controlador.py, modelo.py	Aplicación organizada en tres capas MVC.
Paquete `pruebas`	test_modelo.py, test_controlador.py, test_vista.py	Pruebas automáticas con unittest, una por capa.

6. Interfaz Lo que cada módulo ofrece a los demás y cómo se conectan:

Módulo	Importa	Ofrece
main.py	`from calculadora.vista import Vista`	Ejecuta `Vista().iniciar()`.
vista.py	`tkinter as tk`, `Controlador`	Clase *Vista* con *calcular(operacion)* e *iniciar()*.
controlador.py	`math`, `Calculadora`	Clase *Controlador* con *operar(operacion, texto1, texto2)*.
modelo.py	nada	Clase *Calculadora* con *sumar* y *multiplicar*.

7. Descripción de la arquitectura (vista del modelo de implementación).



8. Integración de construcciones.

Construcción	Se integra	Pruebas ejecutadas	Resultado
B1	modelo.py	test_modelo.py (7 pruebas)	7 de 7 aprobadas
B2	controlador.py	test_controlador.py (9 pruebas)	9 de 9 aprobadas
B3	vista.py	test_vista.py (3 pruebas)	3 de 3 aprobadas
B4 (final)	main.py, README.md	Las 19 pruebas juntas	19 de 19 aprobadas (detalle en el documento de Pruebas)

Así se ve el prototipo integrado, con la suma de 5 y 3 y con la multiplicación de 1000 por 34:

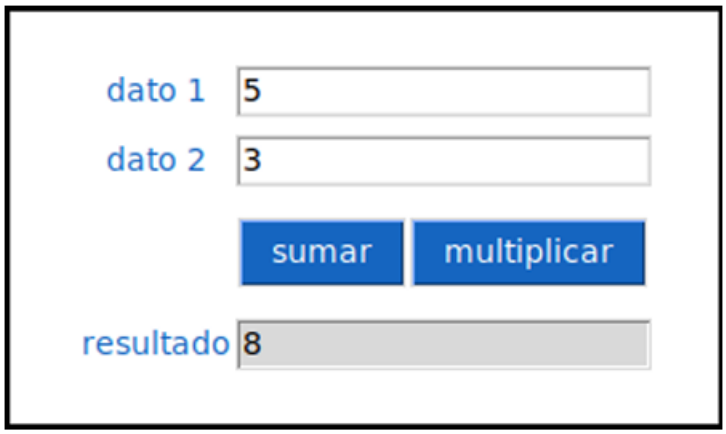


Figura 2. Suma de 5 y 3 (resultado 8).

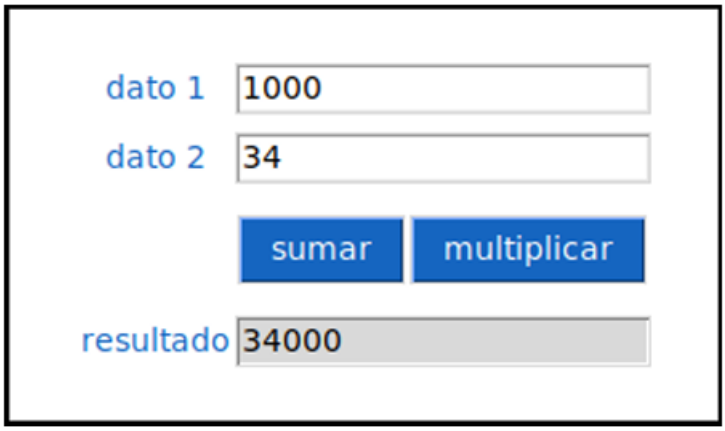
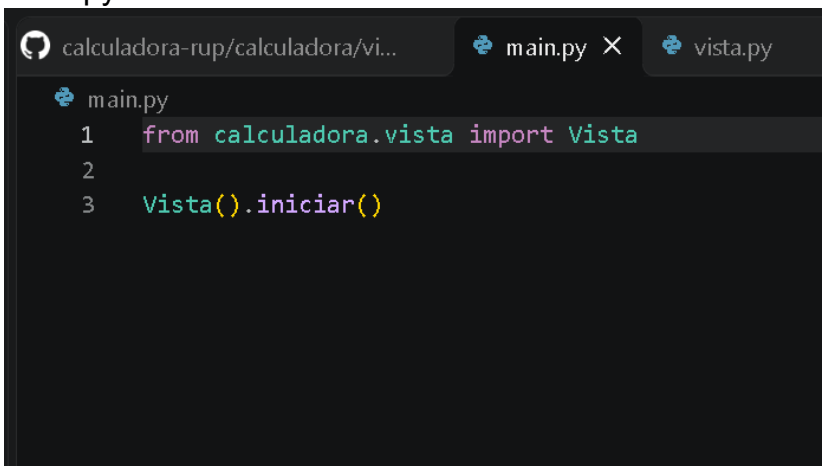


Figura 3. Multiplicación de 1000 por 34 (resultado 34000).

9. Código fuente

main.py



calculadora/modelo.py

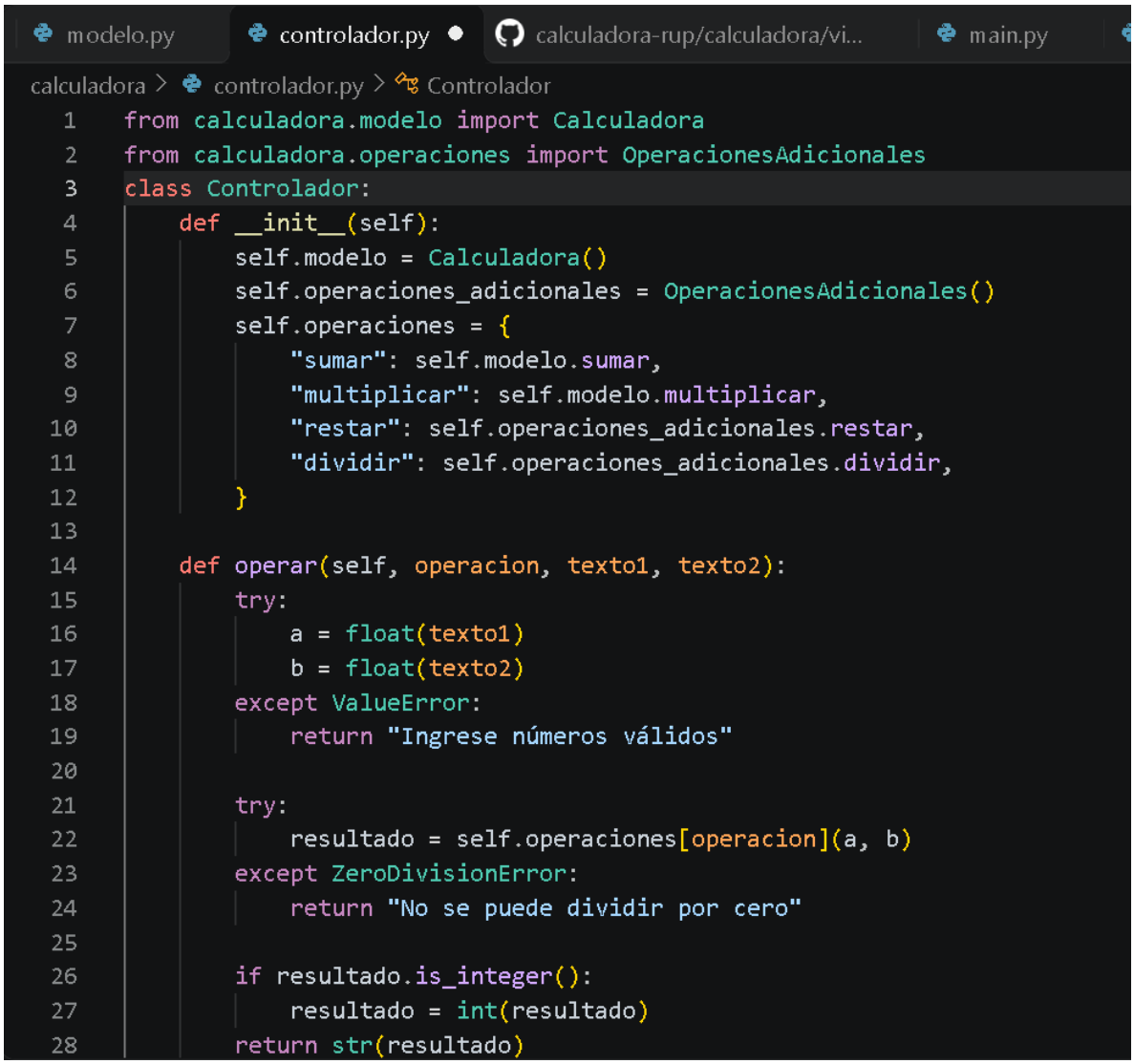
modelo.py controlador.py calculadora-rup/calculador

calculadora > modelo.py > Calculadora > multiplicar

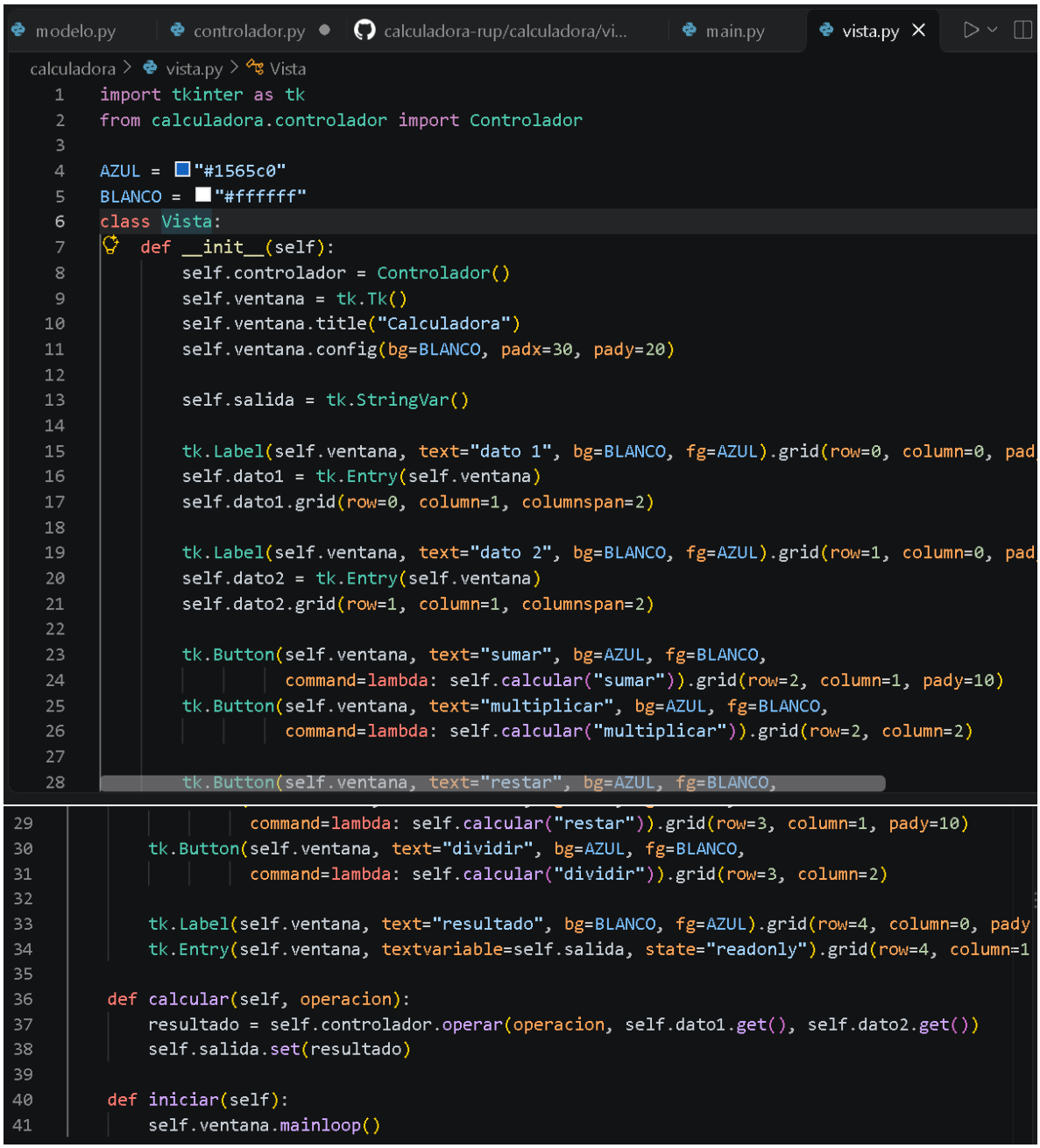
```python
class Calculadora:
    def sumar(self, a, b):
        return a + b

    def multiplicar(self, a, b):
        return a * b
```

Calculadora/Controlador.py



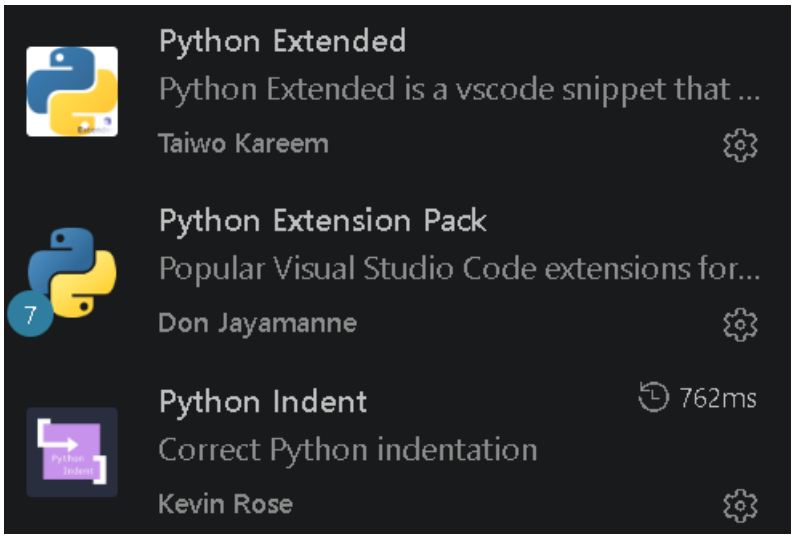
calculadora/vista.py



10. Cómo replicar el entorno

• Instalar Python 3 (marcando «Add Python to PATH») y Visual Studio Code con la extensión de Python.





• Descargar o clonar el repositorio y abrir la carpeta calculadora-rup en VS Code.

• Abrir la terminal de VS Code en esa carpeta y ejecutar python main.py.

• Para las pruebas: python -m unittest discover -s pruebas -t .

Para un flujo de trabajo que permita comunicación con Git-Hub se recomienda.

